BIOBuzz robot: controls, setup and checklists

Last updated 2026-11-11. This one file replaces all the earlier notes.

Status of the code: everything described here was written WITHOUT the robot. It builds against the real FTC SDK 12.0, but it has never been run on real hardware. Part C is the list of things to do, in order, to get from here to a working robot.

No more editing code to tune. Every number in this guide is changed on the robot from the Driver Station and saved on the Control Hub (`/sdcard/FIRST/biobuzz-settings.properties`). The next OpMode you run uses it. You only build and deploy once. The numbers written in the code are just the starting defaults. See A6.

All file names below are under `TeamCode/src/main/java/org/firstinspires/ftc/teamcode/`.

Part A. Controls

A1. Autonomous: `BIOBuzz Auto (pick alliance + side)`, during INIT, gamepad 1

Control	Choice
X / B	Blue / Red
Dpad Up	UP side: our basket is raised, shoot straight away
Dpad Down	DOWN side: our basket is down, wait for the other side to tip it
Dpad Left / Right	Plan: shoot + flower + park / shoot + park / park only

Read the screen before pressing Play: alliance, side, plan, which start mark, camera on/off, flower arm on/off. The choices are remembered until the Robot Controller app restarts.

A2. TeleOp, during INIT (all four TeleOps), gamepad 1

Control	Choice
X / B	Blue / Red alliance (only matters for camera aiming)

A3. Two players: `TeleOp_BaseCode` and `TeleOp_BaseCodeSpeedAdjust` (identical)

Gamepad 1, driver

Control	Action
Left stick	Drive forward/back and strafe
Right stick X	Turn
Dpad Right	Drive speed 25 / 50 / 75 / 100 %
Left trigger (hold)	AIM: turn to our UP cell's aim point and set speed from distance. With the left stick still, the robot's spot stays locked (it cannot be pushed off it) while it turns.
Right stick button (hold)	Bypass position hold

Gamepad 2, operator

Control	Action
Right trigger (hold)	FIRE: balls feed one at a time, each only when the flywheel is at speed
Right trigger released	Stop feeding; flywheel keeps spinning
Y	Spin the flywheel up without feeding
Right bumper	Flywheel off (coasts down)
Dpad Up / Down	Next / previous speed preset (first press leaves AUTO speed)
Dpad Right / Left	Trim speed +25 / -25
X	Clear trim

Control	Action
Left trigger	Intake in
Back	Eject (everything in reverse)
Left bumper	Intake stop
A	Flower arm DOWN
B	Flower arm UP

A4. One player (everything on gamepad 1)

Control	`TeleOp_BaseCodeOneController`	`TeleOp_BaseCodeOneControllerSpeedAdjust`
Left stick	Drive and strafe	same
Right stick X	Turn	same
Left stick button (hold)	AIM	same
Right stick button (hold)	Bypass position hold	same
Dpad Right	Drive speed	same
Right trigger (hold)	FIRE	same
Right trigger released	Stop feeding; flywheel keeps spinning	same
Y	Spin up without feeding	same
Right bumper	Flywheel off	same
X	Clear trim	same
Left trigger	Intake in	same
Back	Eject	same
Left bumper	Intake stop	same
Dpad Up / Down	Next / previous preset	Trim +25 / -25
Dpad Left	Flower arm toggle DOWN / UP	Next preset
B / A	Trim +25 / -25	Flower arm UP / DOWN

A5. Changing a button

Each is one small method in the TeleOp file, with a comment above it listing alternatives.

What	Method
AIM	`aimButton()`
Fire, spin up, flywheel off	`handleManualShootingControl`
Presets and trim	`handleShooterSpeedControl` (the step size is Robot Settings > Shooter speeds > `Trim step`)
Intake	`handleManualIntakeControl`
Flower arm	`handleFlowerArm()`
Autonomous menu	`init_loop()` in `Auton/BioBuzzAuto.java`

Two things work differently from before: - The fire trigger is **hold to fire**. It used to be tap to start, right bumper to stop. - On the one-player TeleOps, holding the intake trigger no longer bypasses the position hold.

Button changes are the one thing that still needs a code change and a re-upload.

A6. Setup and tuning OpModes (change settings without code)

`Robot Settings (edit + save)`, gamepad 1. Works in INIT; nothing drives.

Control	Action
LB / RB	Previous / next group (Shooter PIDF, Shooter speeds, Shooter gate, Intake, Flower arm, Camera, Aim, Position hold, Auto ...)
Dpad Up / Down	Pick a setting (hold to scroll)
Dpad Right / Left	Change it (hold to repeat). ON/OFF settings toggle. **Saved immediately.**
A / B	Bigger steps (x10) / smaller steps (/10)
X	Put this setting back to the code default
Back + Start, hold 2 s	Put EVERY setting back to the code defaults

A `*` after a value means it differs from the code default. After Play, while the flower arm's `Up position` or `Down position` is selected, the arm servo moves to that value as you change it.

`Shooter Tuner (kS, kV, kP)`. Gamepad 1 drives as in A3. All gains start at 0.

Follows the video "PID Control Full Course: Build & Tuning for Robotic Flywheels" (Brogan Pratt): power = **kS** (just gets the wheel turning) + **kV** x target (holds the speed) + **kP** x error (fixes drops). No kI or kD on a flywheel.

Gamepad 2	Action
Dpad Left / Right	Step: kS, kV, kP. The flywheel is driven the way that step needs (kS = only kS; kV = kS + kV, no kP; kP = everything)
Dpad Up / Down	This step's gain + / -
A / B	Steps x10 / /10
Y	Flywheel start / stop
X	Target HIGH / LOW (LOW = 55 % of the preset, like 1500 -> 800 in the video)
RB	Next preset (HIGH)
Left stick button	kV step only: kV helper (kV x target / 1 s average speed)
Right stick button	All gains to 0
Hold RT	Fire a real ball (kP step)
LT / Back / LB	Intake in / eject / stop
Start	**SAVE** kS, kV, kP. Unsaved changes are dropped when the OpMode stops.

Screen (5 lines): what to do now, the three gains and step size, target / speed / error / power, `Last change: back in 0.24 s, overshoot +22`, save status.

`Shot Table Builder`. Gamepad 1 drives as in A3 (hold LT = aim). Gamepad 2 has the A3 shooter and intake buttons, plus:

Gamepad 2	Action
Right stick up / down	Aim higher / lower in the CELL opening (0.05 per push, saved)
Right stick button	Scale the shot model so it gives the current speed at this distance (saved)
A	Record a row: the distance now and the flywheel target now (saved)
B	Delete the row nearest the distance now
Back + Start, hold 2 s	Delete all rows

Screen (5 lines): which CELL and how many tags, distance and aim height, speed now / model / final, the two tuning values, the rows.

`Tune Everything (drive + shoot + settings)`: all of the above in one, for practice-field tuning

Gamepad 1 drives exactly as in A3 (hold LT = AIM). Gamepad 2:

Always	**Hold RT = fire, LT = intake, Back = eject, Y = spin up, Start = switch SHOOT / TUNE mode**
SHOOT mode	RB = flywheel off, LB = intake stop, Dpad Up/Down = preset, Dpad Right/Left = trim, X = clear trim, A = record a row, B = delete the nearest row, right stick up/down = aim height, right stick button = scale the shot model here, left stick button = clear the hold's peak number
TUNE mode	The Robot Settings menu: LB/RB = group, Dpad Up/Down = pick, Dpad Right/Left = change, A/B = step size, X = code default. Shooter, intake, aim, aim point, shot model and position hold changes take effect immediately (no restart). Autonomous and flower arm settings apply next run.

Screen: mode line, flywheel line, 2 aim lines, a `Hold:` line (status, how far off its spot, biggest push), and in TUNE mode 5 menu rows. To turn the flywheel off while in TUNE mode, press Start (back to SHOOT), then RB.

`Tune Position Hold`, gamepad 1

A = hold this spot, B = release, Dpad Up/Down = pick, Dpad Left/Right = change, bumpers = step size, Y = clear numbers, Start = next test step, **X = SAVE**.

Part B. Reference

B1. The switches

Everything optional is OFF until you turn it on. With a switch off, that part does nothing at all. All of them are in `Robot Settings`. In the simulator, the flower arm and the camera are always on.

Switch (Robot Settings group > setting)	Turns on	Turn on at
Flower arm > `Enabled`	The flower arm servo, in TeleOp and autonomous	Step 7
Camera > `Camera enabled`	Limelight: AIM button, speed from distance, tip detection in autonomous	Step 8
Auto camera > `Use camera`	Lets autonomous use the camera (already ON; needs the switch above too)	leave
Auto camera > `Camera fix position`	Autonomous corrects its odometry position from the basket tags at the shot mark	Step 9.6

B2. Where the numbers live

What	Where
Flywheel gains (kS, kV, kP)	`Shooter Tuner (kS, kV, kP)` (or Robot Settings > Shooter PIDF)
Presets and trim step	Robot Settings > Shooter speeds
Speed window, shot detection, feed timing	Robot Settings > Shooter gate
Windmill / transfer speeds, gate hold, jam recovery	Robot Settings > Intake
Flower arm positions	Robot Settings > Flower arm (the servo name `flowerArm` is set in the robot configuration)
Camera pipeline, tag ids, camera mount	Robot Settings > Camera
Aim gains	Robot Settings > Aim
Aim point height, shot model scale, table rows	`Shot Table Builder` (or Robot Settings > Aim point / Shot model)
Launch angle, exit height	Robot Settings > Shot model (measure on the robot)
Position hold	`Tune Position Hold` (or Robot Settings > Position hold)
Autonomous settings and assumptions	Robot Settings > Auto assumptions / Auto shooting / Auto camera / Auto DOWN side / Auto flower / Auto timing
Start marks, shot marks, parking slots	**Still code:** `Auton/cycles/CycleRoute.java`, `DelayedBasketTwoRoute.java`

The saved file is `/sdcard/FIRST/biobuzz-settings.properties` on the Control Hub. Deleting it (or holding Back + Start in Robot Settings) puts everything back to the code defaults. A new setting is added in

`subsystems/RobotSettings.java`, one line each.

B3. What the code does now, in short

- **One ball at a time.** A ball is fed only while the flywheel is within target -40 to +80 ticks/sec. The moment a ball goes through and the speed drops, the gate closes: windmill stopped with BRAKE, transfer stopped. It reopens as soon as the speed is back. The only fixed delay is 0.05 s.
- **The old reverse logic.** While intaking, the windmill still turns slowly backwards to keep balls out of the shooter (unchanged). The old code also reversed it between every shot, which pushed the next ball away; that is what the brake-hold replaces. If a braked windmill does not hold a ball, step 3 says what to change.
- **Speed.** Four presets (1500, 1600, 1700, 1900) each with its own remembered trim. With the camera on and a shot table filled in, AIM switches the speed to AUTO (from distance); a preset button switches back.
- **Jam recovery.** Windmill bogged for 0.3 s: windmill and transfer back up for 0.1 s and retry, three times, then the screen shows FEED JAMMED.
- **One autonomous program** with a menu. The ten older programs are hidden, not deleted: each has `@Disabled` above its class in `Auton/`. Delete that line to bring one back.

B5. Desktop simulator

The simulator in the folder next to this project (`BioBuzSim`) runs this code on a Mac or Windows PC. How to start it, load this project into it and drive it from the keyboard is in `BioBuzSim/MAC-AND-TEAMCODE.md`.

- In the simulator the flower arm and the camera features are switched on automatically (`false || InSimulator.YES` on their switch lines). On the robot they stay off until you change `false` to `true` in steps 7.4 and 8.3.
- Use it for: does it initialize, the order of steps, timing and parking, paths around the frame, the autonomous menu, driver practice.
- Do not take from it: shot speeds, the shot table, flywheel gains, feed timing. Its robot numbers are estimates. Steps 2 to 5 below still need the real robot.

B4. Still needs a code change and re-upload

- Gamepad button mapping (A5).
- Start marks, shot marks and parking slots (9.1).
- Which end of the field carries which four tag ids (9.2 row 7).
- The autonomous menu defaults (`AutoConfig.DEFAULT_*`). They are picked at INIT anyway.
- The hardware names in the robot configuration (`flowerArm`, `limelight`, the motors). These are changed on the Driver Station, not in code.

B4b. Not built

- Odometry is not used to keep the camera distance when the tags are out of view (the last reading is kept 1 s).
- Fire is not blocked while the robot is moving.
- No gamepad rumble.
- The aim point is not corrected when only some of a basket's four tags are visible.

- The final autonomous position is not passed to TeleOp.
- The older moving-shot programs were left as they were (and are hidden).

Part C. TODO, in this order

#	Section	Why this order	Needs
1	Build and first power-on	Nothing else can be trusted until it builds and the basics move	robot on blocks
2	Ball path check	Tells you whether sticking is mechanical before you tune software	balls
3	One-ball-at-a-time gate	The most important behaviour: no slow shots	balls, a target
4	Flywheel tuning	Sets how fast you can shoot	balls
5	Feed timing	Trims the remaining delays, only after 3 and 4 are right	balls
6	Jam recovery	Safety net; quick to check	balls
6b	Position hold	AIM locks the robot's spot with it; tune before camera aiming	open floor, a teammate robot
7	Flower arm	Independent of shooting; needed before the autonomous flower step	servo, a flower bed
8	Camera aim and speed	Builds on working manual shooting	field tags, tape measure
9	Autonomous	Uses everything above; do it last	full field

Tick the boxes as you go. If a step fails, fix it before moving on.

---

1. Build and first power-on

- ☐ **1.1 Build.** Open the project in Android Studio and build.
- ☐ **1.2 Deploy** to the robot. This should be the last deploy needed for tuning: everything after this is done in `Robot Settings` and the tuning OpModes (A6). Put the robot on blocks, wheels off the ground, no balls.
- ☐ **1.2b** Run `Robot Settings`. The bottom line should show `File: /sdcard/FIRST/biobuzz-settings.properties`. Change any value and back again. The line should say `Saved.`
- ☐ **1.3** Start `TeleOp_BaseCode`. INIT screen should show `Camera aim/speed: OFF`.
- ☐ **1.4 Spin up.** Press gamepad 2 **Y**. The flywheel spins up, status shows `Spinning`, nothing else moves.
- ☐ **1.5 Off.** Press gamepad 2 **right bumper**. The flywheel coasts down (it used to brake). Confirm coasting is acceptable.
- ☐ **1.6 Fire, no balls.** Hold gamepad 2 **right trigger**. Status goes `Starting` then `Shooting`; `Feed step` goes to `FEED` (`CLEAR` and `WINDMILL` only show if their times in Shooter gate are above 0).
- ☐ **1.7 Release** the trigger. Feeding stops, flywheel keeps spinning.
- ☐ **1.8 Speed buttons.** Dpad Up/Down change the preset; Dpad Right/Left change the trim by 25; X clears it. The `Shot preset` line follows, e.g. `2/4 1600 +25 = 1625`.
- ☐ **1.9 Motors.** Note which hub each of the four mechanism motors is on (useful if the loop is slow).

2. Ball path check

- ☐ **2.1 Dead-zone test.** Run `TeleOp_IndividualMotorToggle`. Turn the roller, transfer and windmill forward, shooter OFF. Drop in ONE ball. Repeat ten times.
- Goes all the way through every time: any sticking you saw before was software or timing.
- Stops in the same place more than once: that place is a mechanical dead zone (the ball is touching no roller). Write down where. It needs more squeeze, closer rollers or a guide; software cannot fix it.
- ☐ **2.2** Windmill speed assumption: with no ball, feeding at the 1500 preset, the windmill should turn at roughly 90% of the flywheel speed (see `TeleOp_TestBaseCodeSpeeds` for windmill RPM). If it is far lower, the jam detection in step 6 will trigger when there is no jam.

3. One-ball-at-a-time gate

Goal: no ball reaches the flywheel while it is below speed.

- ☐ **3.1** Load four balls. Hold the fire trigger. Film it in slow motion if you can.
- ☐ **3.2** After each shot the windmill must STOP until the flywheel has recovered. No second ball may follow straight behind the first.
- ☐ **3.3** All four balls should land in the same place.

If something is wrong, change it in `Robot Settings`:

You see	Group > setting	Default	Try
Balls are fed but `Balls counted` in autonomous stays low, or a ball follows while the speed is still down	Shooter gate > `Shot drop`	45	40
The gate closes with no ball going through	Shooter gate > `Shot drop`	45	60
A second ball slips through the stopped windmill	Intake > `Hold reverse RPM`	0	150, 300 (careful: reversing here can push balls back out of the intake)
Later balls fall short	Shooter gate > `Resume below`	40	25
Balls go long just after spin-up	go to step 4 (overshoot)		
Never double-feeds and you want faster follow-up shots	Intake > `Transfer hold pwr`	0	0.15, then re-check 3.2

4. Flywheel tuning

Use `Shooter Tuner (kS, kV, kP)` (controls in A6). It follows the flywheel video exactly: tune **kS**, then **kV**, then **kP**. All gains start at 0 and nothing is kept until **gamepad 2 Start**. The code also scales the feedforward for battery voltage, so a tune done at 12.5 V still holds at 12.0 V.

- ☐ **4.1 kS (static).** Dpad Left/Right to `kS`. Press **Y** (the wheel gets ONLY kS). Raise kS (Dpad Up; A/B for step size) until the wheel **just starts to turn from stopped**. Stop (Y), start again, check. Do it 3 times (motor

cold and warm) and use the average. Typical: 0.10 to 0.20. Don't chase perfection.

- ☐ **4.2 kV (velocity feedforward).** Dpad Right to `kV`. Y to run (kS + kV only, no correction). Raise kV until the speed sits **just under** the target (a few ticks/sec under is right; kP finishes it). The **left stick button** does the maths for you: press, wait 1 s, press again until close. Press **X** for LOW: it should also be close (it will not be perfect, that is normal).
- ☐ **4.3 kP (proportional).** Dpad Right to `kP`. Y to run. Press **X** to jump LOW / HIGH again and again (it is how a ball hitting the wheel looks). Watch `Last change: back in X s, overshoot Y`:
- Slow to get back: raise kP.
- Overshoot more than about 30, or you **hear it hum / oscillate**: lower kP.
- Aim for the highest kP that gets back fast without overshooting. The video got 1.1 s -> 0.25 s.
- Then hold **RT** with balls loaded: each shot should recover just as fast.
- ☐ **4.4 Save:** gamepad 2 **Start** (`SAVED kS .. kV .. kP ..`). Every OpMode uses these from now on. (`Tune Everything` > TUNE > **Shooter PIDF** can touch them up later while shooting normally.)
- ☐ **4.5** Write down the recovery time after one real ball: _____ s.
- ☐ **4.6** Repeat one four-ball volley with the battery near 12.2 V. The landing spot should not move.
- ☐ **4.7 Preset speeds.** Change the four preset speeds to what actually scores: Robot Settings > Shooter speeds.
- ☐ **4.8 Time four balls.** Ten volleys, first ball to last ball. Worst time: _____ s. You need this number in step 9.

5. Feed timing

Only after steps 3 and 4 are right. Change one thing at a time and re-check step 3.

All in `Robot Settings`.

Group > setting	Default	Try	Stop if / when
Shooter gate > `Clear reverse s`	0	0.08, 0.15	only if balls are tight against the windmill at the start of a volley (it reverses the windmill and transfer; at 0 it is skipped)
Shooter gate > `Windmill spinup s`	0	0.05, 0.10	only if the windmill stalls on the first ball
Shooter gate > `Windmill resume s`	0	0.03	only if the windmill stalls on balls 2 to 4
Intake > `Transfer intake pwr`	0.35	0.25 or 0.45	balls are pushed past the reversing windmill while intaking (lower) / do not climb (higher)
Intake > `Windmill feed ratio`	0.90	0.8 to 1.0	balls pile up or the flywheel dips more
Intake > `Transfer amp limit`	6.0	the real stall current shown in `TeleOp_TestBaseCodeSpeeds`	

6. Jam recovery

- ☐ **6.1** Cause a jam (two balls loaded tight against the windmill). Expected: the transfer pauses; after 0.3 s the windmill and transfer back up for 0.1 s and try again, up to three times; then the screen shows `FEED JAMMED`. Clear it with Back (eject).
- ☐ **6.2** If it backs up when there is no real jam: in Robot Settings > Intake, raise `Unjam after s` (0.30) or lower `Pause below frac` (0.50).

6b. Position hold (resisting pushes)

The hold keeps the robot on its spot when the sticks are let go, and while AIM is held. It can only push back as hard as the wheels grip: a stronger push makes the wheels slide whatever the settings (more weight, grippier wheels or a rubber anchor pad are the only ways past that). Tuning makes it snap back fast without shaking.

Use `Tune Position Hold` (step-by-step guide on its screen), or `Tune Everything` > TUNE mode > group **Position hold** while watching the `Hold:` line.

- ☐ Open floor, 1 m clear around the robot. Let go of the sticks: `Hold: Holding`.
- ☐ **Hand push:** push the robot sideways firmly for 2 s, let go. Watch `peak` and how fast `off` returns to 0.
- Shakes or rocks back and forth after you let go: raise `Damping KD s` (+0.01), or lower `Stiffness` (-0.25).
- Gives way more than about 3 in, or comes back slowly (over 1 s): raise `Max hold power` up to 1.0, then `Stiffness` (+0.25).
- Gives way slowly during a long push: raise `Integral KI` (+0.25).
- ☐ **Robot push:** a teammate drives into the side for 3 s. Peak under about 4 in, back on the spot after. If the wheels skid while it gives way, that is grip, not tuning: stop here.
- ☐ **AIM push:** hold G1 LT with the camera seeing the UP cell (needs step 8 for the camera part): the screen shows `Aiming, spot locked`. Push it: it returns to the spot and keeps pointing at the cell.
- ☐ Changes are saved as you make them (`Tune Position Hold`: press **X** to save).

7. Flower arm

7.1 Plug it in

- ☐ A positional servo with **270 degrees** of travel (not a continuous one). The arm is a horizontal bar across the front on two side bars: servo 0 = bar DOWN at the front bottom, servo 1 = swung all the way UP (270 degrees) to rest on top of the robot, clear of the shooter.
- ☐ Any free **servo port** on the Control Hub or Expansion Hub. Use a servo power module if the arm is heavy or the servo is high-torque.

7.2 Name it

- ☐ Driver Station, Configure Robot, the hub, Servos, the port you used: type **Servo**, name `flowerArm` exactly. Save and activate.

7.3 Find the two positions

- ☐ Take the arm off the servo horn, or make sure it can swing freely both ways.

- ☐ Run `Robot Settings`, press **Play**, go to the **Flower arm** group (LB/RB). Select `Up position` or `Down position`: the servo moves to that value as you change it (Dpad Right/Left, B for finer steps). Each value is saved as you go. (`Enabled` does not need to be ON for this.)

Arm	Robot Settings > Flower arm >	Default
UP, resting on top, clear of the shooter, inside the size limit	`Up position`	1.00
DOWN, bar at the front bottom	`Down position`	0.00

- ☐ If the arm moves the wrong way, swap the two numbers. If the servo buzzes at an end, move that number in a little (0.03 / 0.97).
- ☐ Time the swing from UP to DOWN with a stopwatch: _____ s. Put that plus 0.2 in Robot Settings > Auto flower > `Arm down s` (default 1.2).
- ☐ The bar swings over the front of the robot, through the path the balls take out of the shooter. Do not fire while the arm is moving; the code does not stop you.

7.4 Turn it on

- ☐ Robot Settings > Flower arm > `Enabled` = ON.
- ☐ In TeleOp the arm does not move at INIT or Play; it first moves when you press its button (A down, B up on gamepad 2; see Part A for one-player).
- ☐ UP is inside the robot's starting size. DOWN does not touch the floor, the intake or the wheels.
- ☐ The servo does not buzz or get hot at either end. If it does, move that position slightly off the stop.

7.5 Use it in TeleOp

- ☐ Stop about a foot from the flower bed, facing it. Arm DOWN. Hold the intake trigger and drive in slowly. When the balls are in, back out, arm UP.
- ☐ While doing this, note the answers to these. You need them in step 9.2:
- Which end of the robot goes in: front (intake end) or back? _____
- Does the arm have to be down before you drive in, or once you are against the flower? _____
- How far back does the robot need to be for the arm to swing down freely? _____ in
- How long do the balls take to come out and get taken in? _____ s
- Do balls keep coming out while you back away? _____

8. Camera aim and speed from distance

Do this only when manual shooting (steps 3 to 5) works. All settings are in `Robot Settings`, groups **Camera** and **Aim**. The shot table is made with `Shot Table Builder`.

8.1 Limelight: plug in, configure, set up the pipeline

Plug in - ☐ Mount the Limelight 3A rigidly, looking toward where the baskets will be, with nothing on the robot in its view. It must not wobble: every degree of movement changes the distance reading. - ☐ Plug it into a **USB 3.0 port on the Control Hub** (the blue one) with a short cable, tied down so it cannot pull out. It is powered through USB; nothing else to wire.

Robot configuration - ☐ Driver Station, Configure Robot, **Scan**. The camera appears as a device. Name it `limelight` exactly (the code looks for that name). Save and activate.

Open the Limelight web page - ☐ Unplug the Limelight from the robot and plug it into a laptop by USB. Open `http://limelight.local:5801` in a browser. (It can also be reached through the Control Hub's Wi-Fi; the address for that is in Limelight's FTC documentation.)

Create or check the AprilTag pipeline - ☐ Pick a pipeline slot and set its type to **AprilTag (Fiducial)**. Note the slot number (0, 1, 2 ...). - ☐ Tag family: **36h11** (the family FTC field tags use). - ☐ **Tag size:** enter the printed size of the basket tags from the BIOBUZZ game manual, or measure the black square on a real tag. A wrong size scales every distance wrong. - ☐ Turn on 3D / "Full 3D" targeting. The distance code needs each tag's 3D position; without it the Driver Station shows `Distance (in): unknown`. - ☐ No ID filter, or one that includes all of 30 to 45. - ☐ Exposure: bright and sharp enough that all four tags on a basket are outlined steadily from your farthest shooting spot, but low enough that they do not blur when the robot turns. Adjust while watching the live picture. - ☐ Resolution: if far tags flicker in and out, raise it (the frame rate drops; that is acceptable here). - ☐ Save the pipeline. Plug the Limelight back into the Control Hub.

Tell the code - ☐ Robot Settings > Camera > `Pipeline` = the slot number (default 1). TeleOp and the new autonomous both use this one number.

One pipeline serves both alliances; the code picks the alliance's tags itself. You do NOT need to upload a field map: the code works from the tags' positions relative to the camera, not from a field position.

8.2 Measure the camera mount

Tape measure and a phone level app. These go in Robot Settings > **Camera** (all in inches and degrees). Measure again any time the camera is moved or bumped.

Measure	How	Setting	Default
Pitch	Phone flat on the camera's front face; angle above level. Robot on a level floor.	`Pitch deg`	43
Forward offset	Robot centre (centre of the four wheels) forward to the lens	`Forward in`	7
Side offset	Lens right of the centre line (left is negative)	`Right in`	0
Height	Floor to lens centre	`Height in`	10

8.3 Turn it on

- ☐ Robot Settings > Camera > `Camera enabled` = ON. Leave the shot table empty for now.
- ☐ INIT screen: pick the alliance (X / B). It should show `Camera: found` and `Auto speed: model, 0 table rows`. If not found, the robot configuration needs a device named `limelight`.
- ☐ Press Play and point at one of your baskets: the screen shows `Basket: tags NN-NN`, `Aim error (deg)` and `Distance (in)`.

8.4 Check the tag ids

- ☐ Point at each of your two baskets. Expected: blue 38-41 and 42-45, red 30-33 and 34-37.
- ☐ If the tags are in the picture but the screen says "not in view", read the real ids in the Limelight web view and correct Robot Settings > Camera > `Blue first tag` / `Red first tag`.
- ☐ Write down which tag range is on which basket. You need it in step 9.2.

How the camera aims (read once)

- Only our alliance's tags are used: red 30-37, blue 38-45. Each CELL has 4 tags in a row on its bottom face.
- The UP cell (the one to shoot into) is picked by how high its tags are (Auto DOWN side > `Basket up tag in`).
- From ANY visible tag, the code knows the cluster centre (each tag is a fixed -6.5 / -2.75 / +2.75 / +6.5 in from it), and from the game manual the CELL opening (7.19 in out from the cluster, 14 in tall, HIVE tilted 30°).
- The **aim point** is dead centre left-right in the opening, at **Height in opening** (0 = bottom lip, 1 = top). Turning, distance and speed all use this point.
- Speed = **shot model** (projectile physics from launch angle and exit height, scaled to our flywheel) plus a correction from any **recorded rows**. At a recorded distance the speed is exactly what was recorded.

8.5 Check the distance

Use `Shot Table Builder`. The `(tags NN)` number is the floor distance from the robot centre to the tag cluster centre: that is the one to tape-check.

- ☐ Tape three marks, for example 30, 54 and 78 in along the floor from the point under the middle of the tag sticker. Put the robot centre on each, square to the CELL.

Tape (in)	Screen (in)	Error

- ☐ Same error at all three: change Camera > `Forward in` by that amount. Restart the TeleOp to re-check.
- ☐ Error grows with distance: Camera > `Pitch deg` is off; change 1 degree at a time.
- ☐ Done when all three are within about 1 inch. It must be repeatable more than perfect.

8.6 Check the turn direction

- ☐ Park about 15 degrees to one side with the tags in view. Hold AIM.
- ☐ Turns toward the basket: fine. Turns away: Aim > `Aim direction` = -1.

8.7 Tune the aim

Follows the video "How To Auto-Rotate to an AprilTag with Mecanum Drive" (Brogan Pratt): a PD controller on the angle to the target, output clamped (`Aim max power`, 0.4), zero inside `Aim stop deg`. The same PD turns the robot while driving AND while the spot is locked. Aim strength no longer depends on the drive speed.

Easiest in `Tune Everything` > TUNE mode > group **Aim** (changes apply at once): hold AIM with G1 LT, change values with G2, try again.

- ☐ **kD first at 0.** Set `Aim kD` = 0 (X = code default is 0).
- ☐ **kP:** from about 20 degrees off, hold AIM. Raise `Aim kP` until it **swings past** the target, then lower it until it stops just on target without passing it. It should still turn quickly.
- ☐ **kD:** now add `Aim kD` in tiny steps (B for 0.0001 steps). A little damping lets it arrive fast and settle. Too much: it **jitters back and forth**; go back down.

- ☐ `Aim stop deg` (0.5): the robot stops turning inside this error. Raise to 1.0 if it buzzes when lined up. Far shots need it smaller; a wide opening allows larger.
- ☐ `Aim max power` (0.4): lower if it still whips past from far off; raise only if turning is too slow.
- ☐ `Aim min power` (0): only if it stops just short of the target, try 0.05.
- ☐ Check while driving: hold AIM and strafe/drive. It should keep pointing at the cell.
- ☐ Target: `AIMED` within about half a second, no back-and-forth, both still and while driving.

8.8 Left/right

The aim point is always dead centre on the tags; there is no left/right offset. If balls miss sideways while `AIMED`, the cause is turning (8.7, 8.13) or the shooter itself not firing straight.

8.9 Measure the shooter (tape and phone level, robot on the floor)

Robot Settings > **Shot model**: - ☐ `Launch angle deg`: angle of the ball's path as it leaves the shooter, above level (phone level along the exit ramp or hood). Default 55 is a guess. - ☐ `Exit height in`: floor to where the ball leaves the shooter. Default 12 is a guess. - ☐ `Exit forward in`: how far that exit point is in front of the robot centre (negative = behind).

8.10 Tune the aim height and model scale (Shot Table Builder or Tune Everything SHOOT mode, no rows yet)

Start with **no rows** (hold Back + Start 2 s if there are some). For each shot: hold G1 LT until `AIMED`, set the speed with presets and trim (G2 Dpad), fire 5 balls.

- ☐ **Near spot** (closest you will really shoot from). Find the speed that drops the ball in the middle of the opening. Press **G2 right stick button**: the model is scaled to that speed here.
- ☐ **Far spot** (farthest you will shoot from). Find the speed that works there and compare it with `model` on the screen:
- Your speed is HIGHER than the model: aim point too low for our shot. Push **G2 right stick UP** (higher).
- Your speed is LOWER than the model: push the right stick **DOWN**.
- ☐ Go back to the near spot, press the right stick button again (re-scale), then re-check the far spot. Two or three rounds; stop when both are within about 20 ticks/sec.
- ☐ Middle spot: the model should already be close.

8.11 Record rows (fine correction)

- ☐ At 3 to 5 distances you really shoot from: hold AIM until `AIMED`, find the speed where 8 of 10 score (trim), press **G2 A**. Each row pins the speed there; between rows the model's curve is used.
- ☐ A wrong row: stand at that distance and press **B**.
- ☐ If you later change the aim height or the model a lot, the rows still pin their own distances, but re-check between them.

8.12 Verify auto speed

- ☐ INIT shows `Auto speed: model, N table rows`.
- ☐ Hold AIM: `Shot preset` shows `AUTO 1640 +0 = 1640`.
- ☐ At each measured distance and halfway between two: 8 of 10 score with no manual change.

- ☐ If every distance is slightly short or long, trim fixes it (in AUTO the trim applies at every distance). If you always need it, re-record the rows with the trim in.
- ☐ One distance again near 12.2 V. If shots change, return to step 4.

8.13 Fire tolerance

Aim > `Fire tolerance deg` (2.0): while AIM is held, balls feed only inside this aim error.

- ☐ Misses sideways while `AIMED`: make it smaller (1.0).
- ☐ Hesitates although shots would go in: make it larger (3.0).
- ☐ To turn the aim gate off: Aim > `Aim gate` = OFF.

8.14 How it behaves when set up

- Hold AIM: robot turns to the aim point of our UP cell, speed goes to AUTO (model + rows).
- While AIM is held and the left stick is still, the position hold locks the robot's x/y and only lets it turn toward the aim point: pushes are resisted. Move the left stick to drive (the camera keeps turning it). With no target in view, the right stick turns the robot by hand. Right stick button = hold off completely.
- Turning uses the same aim PD (`Aim kP`, `Aim kD`, `Aim max power`) whether the spot is locked or you are driving.
- Screen while driving: two lines, `Aim 30-33 UP 3 tags err +0.4 AIMED` and `Dist 82 in auto speed 1650`.
- Hold FIRE: balls feed one at a time, at speed, and (while AIM is held) only when aimed.
- Fire without AIM is never blocked by the camera.
- A preset button leaves AUTO; holding AIM again returns to it.
- Camera unplugged: everything works on manual presets.
- Autonomous uses the camera speed only once at least one row is recorded; before that it uses `UP shot speed` / `DOWN shot speed`.

---

9. Autonomous (do this last)

Program: `BIOBuzz Auto (pick alliance + side)`. All settings: Robot Settings, the groups starting with **Auto**.

What it does:

- **UP side:** drive to the shot mark with the flywheel spinning up, aim, shoot, flower (arm, drive in, intake, back out), park.
- **DOWN side:** drive to the shot mark and wait. When the camera sees our basket come up, shoot; then the flower if there is still time; park. If the basket never comes up, it leaves for parking at about 22 to 23 s.
- **Position fix (optional, step 9.6):** at the shot mark the camera can correct the odometry position from the basket tags, so later drives are not thrown off by a small placement error.
- **Always:** every loop it works out the drive time to its parking slot and leaves in time to be parked by 29.5 s. Everything is off at 29.7 s.

Shooting ends when four balls are counted, when the feed has been open 2.5 s with nothing going through, or when the shooting allowance is used up.

9.1 Starting positions

The program does not find the robot at the start: pressing INIT tells the odometry "the robot is exactly on the chosen start mark, facing the chosen way". If the robot is an inch or a few degrees off, every position after that is off by the same amount, so careful placement matters a lot. (Step 9.6 lets the camera take out a small position error at the shot mark, but not a heading error, and not before the first shots.)

The numbers are the position of the robot's **tracking point** (the Pinpoint odometry origin, close to the robot centre), in metres from the centre of the field. Heading 0 points to +x, 90 to +y. The INIT screen shows the same numbers for the side you picked.

Start	Alliance	x (m)	y (m)	Robot faces	Notes
Basket 1 wall mark	Red	-0.360	-1.564	90 deg (away from the wall)	Back of the robot against the wall
Basket 1 wall mark	Blue	+0.360	+1.564	270 deg (away from the wall)	Same, other end of the field
Basket 2 wall mark	Red	-0.245	+1.564	270 deg (away from the wall)	Shifted sideways to clear the flower pot on that wall
Basket 2 wall mark	Blue	+0.245	-1.564	90 deg (away from the wall)	Same, other end of the field

With the default settings the UP side uses the Basket 1 wall mark and the DOWN side the Basket 2 wall mark.

Where it goes from there (red; blue is the same turned half a turn):

Start	Shot mark (m)	Faces	Parking slot (m)
Basket 1 wall mark	-0.349, -1.099	about 88 deg	-1.451, +0.640
Basket 2 wall mark	-0.349, +1.099	about 272 deg	-1.451, +1.257

The 1.564 m figure puts the tracking point about 9.2 in from the wall, which fits an 18 in robot with its back touching the wall and the tracking point at its centre.

- ☐ Measure where your Pinpoint tracking point really is on the robot. If it is not the centre, the robot body sits shifted by that amount from where the code thinks.
- ☐ Mark the start positions on your practice field with tape (wall contact gives y; a tape line gives x).
- ☐ Make a simple alignment aid (a block against the wall, or a line on the tile) so placement is the same every match.
- ☐ Square the robot to the wall before INIT; do not move it after INIT.
- ☐ Before pressing Play, check that the screen's "Place robot" line matches where the robot is standing.

Start marks are still in code (needs a rebuild). To change a start mark: `CycleRoute.Start.wallX()` and `wallPose()` (Basket 1 mark), and `initial` in `DelayedBasketTwoRoute` (Basket 2 mark), both in `Auton/cycles/`. The shot marks and parking slots are in the same two files.

9.2 Check the assumptions

Each row is a guess made without the robot. Check it on the field. If it is wrong, change the one setting named. Settings in **bold** are in Robot Settings (no rebuild). The others are code and need a rebuild.

#	Assumption	How to check	If wrong, change
1	The UP side (basket raised at the start) is the old "Basket 1" wall mark; the DOWN side is the old "Basket 2" wall mark	On the field, see which basket is raised at the start of a match and which wall mark is beside it. The INIT screen shows which mark the chosen side uses and where to place the robot.	**Auto assumptions > Swap start walls = ON**
2a	Flower 1 is on the side wall near the Basket 1 wall; flower 2 is on the Basket 2 wall	Run "Shoot + flower + park" slowly and watch where the robot goes, or compare the coordinates with the field	**Auto assumptions > Flower 1 / Flower 2 x m, y m, facing deg** (red, metres from field centre)
2b	A Basket 1 start uses flower 1; a Basket 2 start uses flower 2	Decide which flower your alliance should use from each start	**Auto assumptions > Flower for B1 start, Flower for B2 start** (1 or 2)
3	The robot goes into the flower front first (intake end)	Your answer from step 7.5	**Auto assumptions > Flower back first = ON**
4	The arm comes down before driving in	Your answer from step 7.5	**Auto assumptions > Arm down before in = OFF** (lowers it once against the flower)
5	The intake runs while driving in, at the flower, and while backing out	Your answers from step 7.5	**Auto assumptions > Intake driving in, Intake backing out, Intake extra s** (seconds of intake after backing out)
6	The flywheel spins for the whole DOWN-side wait	Watch the battery over a few DOWN-side runs	**Auto assumptions > Spin while waiting = OFF** (spins up when the tip is seen; costs a second or two)
7	Tag ids: the Basket 2 end carries the lower four of the alliance's eight, the Basket 1 end the upper four	Your note from step 8.4	**Camera > Blue first tag / Red first tag**. If the two ends are the other way round, that needs a one-line code change in `SideRoute.basketFirstTag`.
8	From the DOWN shot mark the camera can see our basket's tags once the basket is up	Step 9.5 below	Move the shot mark (code), or use the timed shot (**Auto DOWN side > No camera shoot at s**)
9	Each start parks in the half of the parking zone nearer its own wall	Watch a park-only run from each start; agree halves with your partner	Code: `CycleRoute.OUR_PARK_Y_METERS` (Basket 1 start), `DelayedBasketTwoRoute.park` (Basket 2 start)

| 10 | The robot starts exactly on the start mark of 9.1, 18 in long, back against the wall, tracking point at the centre | Step 9.1 | Code: the start mark in `CycleRoute` / `DelayedBasketTwoRoute` | | 11 | Four balls are loaded by hand before the match | | **Auto shooting > Balls** | | 12 | Both shot marks are the same distance from their basket, so one speed suits both | Step 9.4 | **Auto shooting > UP shot speed, DOWN shot speed** | | 13 | The robot must stay outside the metal frame in the middle of the field; it stops if odometry says it is inside | Park-only runs in step 9.3 | Code: `FrameClearance` in `Auton/cycles/` (leave alone unless the robot stops with "Too close to the frame") | | 14 | The basket tags are where the older autonomous code's field model says they are | Step 9.6, step A | **Auto camera > Tag centre x m, Tag row y m, Tag 1-4 offset in** |

- ☐ 1 checked _____
- ☐ 2a, 2b checked _____
- ☐ 3, 4, 5 checked _____
- ☐ 6 decided _____
- ☐ 7 checked _____
- ☐ 9 agreed with partner _____
- ☐ 10 to 13 checked _____
- ☐ 14 checked in step 9.6 _____

9.3 Park only

- ☐ Plan "Park only", each side, each alliance (four runs). The robot leaves its start mark, goes round the frame and stops in its slot. Note the `Parked at` time.
- ☐ The slot it takes leaves the other half free.

9.4 Shoot + park, camera off

- ☐ Set the speeds that score from each shot mark: stand the robot on the mark in TeleOp, use presets and trim, and put the result in Robot Settings > Auto shooting > `UP shot speed` and `DOWN shot speed` (both 1500 now).
- ☐ Set Auto shooting > `Shoot budget s` (13) to your worst time from step 4.8 plus 1.5 s.
- ☐ UP side, plan "Shoot + park": four balls score, `Balls counted` reaches 4, robot parks.
- ☐ DOWN side with the camera off: it shoots at 15 s without seeing the tip (Auto DOWN side > `No camera shoot at s`; -1 means never shoot without a camera). Test with a partner robot, or tip the basket by hand before 15 s.

9.5 Camera in autonomous (needs step 8 done)

- ☐ UP side: the robot turns slightly to line up, then shoots. If it turns away and then shoots from the planned heading after 2.5 s, set Auto camera > `Turn sign` = -1.
- ☐ DOWN side, tip detection. Run the DOWN side and read the `Detail` line while it waits:
- Basket down: tag height _____ in (or "not in view", which is fine)
- Basket up (tip it by hand): tag height _____ in
- Set Auto DOWN side > `Basket up tag in` (43.7) halfway between the two.
- ☐ DOWN side again: tip the basket by hand at about 8 s. The robot should start shooting about 0.7 s later.
- ☐ DOWN side, never tip it: the robot leaves for parking at about 22 to 23 s without shooting.

9.6 Camera position fix (needs 9.5 done)

What it is: while the robot stands at the shot mark looking at the raised basket, the camera works out where the robot really is from the basket's tags and compares that with the odometry. This catches a robot that was placed a little off its start mark, or wheels that slipped on the way. With the fix switched on, the odometry is corrected by the difference, so the drive to the flower and to the parking slot starts from the true position. Only position is corrected; the heading stays the odometry's (squaring the robot to the wall at the start is still what sets the heading).

It does not move the robot back onto the shot mark: the robot keeps shooting from where it is, with the camera aim and distance allowing for that.

It is OFF to begin with and only **shows** the difference, because it depends on numbers that have not been checked on a real field: where the basket tags are.

Step A: check the tag positions - ☐ With a tape measure, on the real field, with a basket UP, measure for the RED side (blue is the same turned half a turn):

Measure	Assumed	Yours	Robot Settings > Auto camera >
x of the middle of the row of four tags, from the field centre (m)	-0.32385		`Tag centre x m`
Distance of the tag row from the field's centre line, y (m)	0.3273		`Tag row y m`
Each tag's position along the row from the middle, in id order (in)	-6.5, -2.75, 2.75, 6.5		`Tag 1 offset in` … `Tag 4 offset in`

Step B: check that the camera and the odometry agree - ☐ Leave Auto camera > `Camera fix position` OFF. - ☐ Place the robot on the start mark as carefully as you can. Run the UP side, plan "Shoot + park". - ☐ After it has shot, read the line `Camera position check` on the Driver Station. It shows `camera - odometry = (x, y) in`. Write it down. Repeat three times, and on the other alliance.

Run	x (in)	y (in)

- ☐ With careful placement the numbers should be small (within about an inch) and the same each run.
- Small and repeatable: go to step C.
- Large but the same every run: a constant error. Check, in this order, the camera forward offset and pitch (step 8.2 and 8.5), the tag positions (step A), and where the Pinpoint tracking point is on the robot (step 9.1).
- Different every run: the camera reading is not steady. Check the mount is rigid, the exposure (step 8.1), and that at least two tags are in view from the shot mark.
- `not measured yet`: the camera did not see at least two of our basket's tags while standing still.
- `frames disagree, ignored` or `too large to believe, ignored`: the safety limits rejected it; treat as "different every run" or "large" above.

Step C: prove it corrects in the right direction - ☐ Still with the switch off, place the robot deliberately 2 in to one side of the start mark. The line should now show about 2 in in one axis. Then 2 in further from the wall: about 2 in in the other axis. (Which sign is which does not matter, as long as the size matches what you did.)

Step D: turn it on - ☐ Auto camera > `Camera fix position` = ON. - ☐ Place the robot 2 in off the mark again. The line should end with `APPLIED`, and the robot should now park in the same spot as when it was placed correctly. Compare with a run with the switch off. - ☐ Watch the first run closely with a hand near the stop button: this is the first time the code changes the odometry while driving. If the robot lurches when `APPLIED` appears, turn the switch off.

Settings (Robot Settings > Auto camera), change only if needed:

Setting	Default	Meaning
`Fix max in`	6	A correction larger than this is not believed and not applied.
`Fix min in`	0.3	A difference smaller than this is left alone.
`Fix min tags`	2	How many of the four tags must be measured in a frame.
`Fix samples`	8	Still frames used (the middle value is taken).
`Fix spread in`	2	The frames must agree within this or nothing is applied.

9.7 Flower in autonomous (needs step 7 done)

- ☐ Set Auto flower > `Standoff in` (12) and `Collect s` (2.5) from your step 7.5 notes.
- ☐ UP side, plan "Shoot + flower + park". Watch: drive to the stand-off point, arm, drive in, intake, back out, arm up, park.
- ☐ If the robot bounces off the flower bed, lower Auto flower > `Push m per s` (0.25).
- ☐ DOWN side with an early tip (by hand at about 5 s).

About time: the whole flower detour is estimated at about 14 s, so it is only started if shooting is finished by roughly 15.5 s on the clock. On the DOWN side that needs an early tip; otherwise the robot shoots and parks, and the status line says "no time for the flower".

9.8 Final runs

- ☐ Each plan you intend to use, five times in a row. `Parked at` must stay under 29.5 s every time.
- ☐ Other settings, change only if needed:

Robot Settings >	Default	Meaning
Auto DOWN side > `Max wait s`	30	Stop waiting for the tip at this time. 30 means "as long as it is still useful".
Auto shooting > `No shot give up s`	2.5	Feed open this long with no ball: assume empty.
Auto camera > `Aim max deg`	8	The camera may turn the robot at most this far from the planned heading.
Auto timing > `Park margin s`	0.5	Extra safety time on top of the calculated drive to the slot.
Code: `AutoConfig.DEFAULT_BLUE`, `DEFAULT_UP_SIDE`, `DEFAULT_PLAN`	blue, UP, full	What the menu shows first (picked at INIT anyway).